DECODING BROWSER NETWORK SYSTEMS

Exploring the Complex Functions Beyond Basic Connectivity in Modern Browsers

Understanding the Browser's Network System 🌐

The network system of browsers like Google Chrome or Mozilla Firefox is intricately responsible for a wide range of tasks beyond merely opening a TCP connection and downloading HTML. Let's delve into these tasks step-by-step.

Main Jobs of the Browser Network System

DNS Resolution

Before any connection can be made, the network system must convert a domain name into an IP address using the Domain Name System (DNS).

Example:

example.com → 93.184.216.34

Without DNS, the browser cannot find the server.

Opening Connections

The network system establishes connections using the Transmission Control Protocol (TCP), managing the opening, reusing, and closing of these connections. Browsers often reuse the same connection for multiple requests to optimize performance.

Secure Communication

For websites that use HTTPS, the network system performs encryption using Transport Layer Security (TLS). This process protects data from potential attackers.

Sending HTTP Requests

The browser sends requests using the Hypertext Transfer Protocol (HTTP).

Examples of requests:

- GET /index.html
- GET /style.css
- GET /script.js

The network system is responsible for building and dispatching these requests.

Downloading Resources

The network layer downloads all necessary resources for the page, including:

- HTML
- CSS
- JavaScript
- Images
- Fonts
- Videos
- API data

Modern browsers often download many resources in parallel to enhance efficiency.

Caching

The network system manages the browser's cache.

Example:

If logo.png was downloaded previously, the browser can load it from the cache instead of the internet, speeding up the website loading process.

Handling API Requests

When JavaScript runs a command like fetch("/api/users"), the request is processed by the browser's networking system rather than being sent directly to the internet.

Flow:

- JavaScript
- ↓
- Browser Web API
- ↓
- Network system
- ↓
- Server

Managing Cookies

The network layer handles cookies, such as Cookie: session=abc123. These cookies are automatically attached to requests and are essential for managing login sessions, authentication, and user tracking.

Handling Redirects

In cases where the server issues a redirect, such as 301 Redirect, the network system automatically follows the redirect.

Example:

http://site.com → https://site.com

🔟 Request Prioritization

Browsers decide which files to download first to optimize loading times.

Example priority:

1. HTML
2. CSS
3. JavaScript
4. Images

This prioritization helps the page load faster and more efficiently.

Understanding Request Prioritization

Why Browsers Need Request Priority

Imagine a page with various resources:

- <link rel="stylesheet" href="style.css">
- <script src="app.js"></script>
- <img src="hero.png">
- <img src="photo1.png">
- <img src="photo2.png">

If resources were downloaded randomly, the page might show no style, have a broken layout, or load slowly. Therefore, browsers prioritize important resources first.

Typical Priority Order

Browsers like Google Chrome generally follow this order:

- **Highest Priority:** HTML document
- **Very High:** CSS files
- **High:** JavaScript files
- **Medium:** Fonts

- **Low:** Images
- **Lowest:** Prefetch/preloads/background resources

Reason:

- CSS is required for rendering the layout.
- JavaScript might block rendering.
- Images are visual elements only.

How the Browser Decides Priority

As the rendering engine (like Blink) parses HTML, it discovers resources and assigns priority.

Example:

- For <link rel="stylesheet" href="style.css">, the browser assigns a **HIGH** priority.
- For <img src="photo.png">, the priority might be **LOW**.

Connection Limits

Browsers limit connections per domain, for instance, to 6 connections per domain. If there are 20 resources, 6 will download while 14 wait in a queue. The network scheduler selects which requests begin first.

JavaScript Can Affect Priority

Scripts can block HTML parsing, so they are treated as high priority.

Example:

- <script src="app.js"></script> is high priority.
- <script src="app.js" defer></script> is lower priority since it can load later.

Visible Images Get Higher Priority

Modern browsers prioritize visible elements on the screen.

Example:

- Hero image (top of page) → **High priority**
- Image at bottom → **Low priority**

This prioritization helps with faster page rendering.

Developers Can Control Priority

Developers can provide hints to the browser.

Example:

- Preload: <link rel="preload" href="font.woff2" as="font">
- Lazy loading images: <img src="photo.jpg" loading="lazy">

This guidance informs the browser to load certain resources later.

Simplified Internal Flow

Internally, the browser processes resources as follows:

- HTML Parser
  - ↓
- Resource discovered
  - ↓
- Priority assigned
  - ↓
- Network Scheduler
  - ↓
- Request queue
  - ↓
- Download

Understanding these intricate processes helps illuminate how browsers efficiently render web pages, ensuring a smooth and swift user experience.